

road-pulse — Plan projekta

Analiza saobraćajnih nezgoda u Srbiji — od sirovih podataka do prediktivnog modela.

Planirani delovi

- 1. **Mapiranje opasnih tačaka (Heatmap)** - najvisuelnija ideja Grupiši nezgode po koordinatama, nađi "crne tačke" gde se nezgode najčešće dešavaju po opštini ili ulici. Stack: Python + Folium/Kepler.gl + PostgreSQL/PostGIS. Ovo je direktno upotrebljivo za grad — mogu videti gde da postave semafore ili znakove.
- 2. **Vremenska analiza rizika** Kada se nezgode najčešće dešavaju - doba dana, dan u nedelji, mesec, sezona. Možeš predviđati "visokorizične periode" i praviti alerting sistem. Odlično za ML + time series analizu.
- 3. **Prediktivni model ozbiljnosti nezgode** Na osnovu tipa nezgode, broja vozila, lokacije i vremena - predvidi da li će biti samo materijalna šteta, povređenih, ili poginulih. Klasifikacioni ML problem sa labelovanim podacima.

Stack

Sloj	Tehnologija
Podaci	data.gov.rs (xlsx)
Baza	PostgreSQL
Backend	FastAPI + SQLAlchemy
Analitika	Pandas, Matplotlib, Seaborn
ML	Scikit-learn
Infrastruktura	Docker, Docker Compose
Verzionisanje	Git + GitHub

Faza 1 — Setup & Infrastruktura

Cilj: Projekat pokrenuti lokalno, baza radi, podaci unutra.

1.1 Struktura foldera

Kreiraj sledeću strukturu ručno:

```
road-pulse/
├── backend/
│   └── app/
│       ├── api/
│       ├── models/
│       ├── schemas/
│       ├── services/
│       ├── db/
│       └── main.py
├── data/
│   ├── raw/           ← xlsx fajlovi idu ovde (nisu na GitHubu)
│   └── processed/
├── notebooks/
│   └── EDA.ipynb
├── scripts/
│   └── import_data.py
├── .env
├── .env.example
├── .gitignore
├── docker-compose.yml
└── README.md
```

1.2 Virtuelno okruženje

```
python -m venv .venv
.venv\Scripts\activate      # Windows
pip install fastapi uvicorn sqlalchemy psycopg2-binary \
    pandas openpyxl python-dotenv
pip freeze > requirements.txt
```

1.3 Docker Compose

Kreiraj `docker-compose.yml` sa PostgreSQL servisom:

```
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: road_pulse
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Pokretanje: `docker compose up -d`

Provera: `docker compose ps` — status treba da bude `running`

1.4 .env fajl

```
DATABASE_URL=postgresql://user:password@localhost:5432/road_pulse
```

`.env.example` isti sadržaj ali bez vrednosti — ovaj ide na GitHub.

✓ **Faza 1 završena kada:** `docker compose up` radi bez grešaka i možeš se konektovati na bazu.

Faza 2 — Podaci & Baza

Cilj: Sirovi `xlsx` podaci čiste i dostupni u PostgreSQL-u.

2.1 SQLAlchemy model

U `backend/app/models/accident.py` definiši tabelu `accidents` sa kolonama:

- `id` (primary key, auto)
- `acc_id` (original ID iz dataseta)
- `police_dep` , `municipality`
- `date_time` (DateTime)
- `longitude` , `latitude` (Float)
- `acc_type` , `severity` , `description`
- `created_at` (auto timestamp)

2.2 Import skripta

U `scripts/import_data.py` napiši skriptu koja:

1. Čita xlsx sa `pd.read_excel()`
2. Čisti kolone (rename, parsiranje datuma, strip whitespace)
3. Proverava duplikate po `acc_id`
4. Ubacuje u bazu batch insert-om (`df.to_sql()` ili SQLAlchemy bulk)
5. Štampa: Uvezeno X redova, preskočeno Y duplikata

Pokretanje: `python scripts/import_data.py`

2.3 Validacija podataka

Posle importa proveri u bazi:

```
SELECT COUNT(*) FROM accidents;
SELECT COUNT(*) FROM accidents WHERE latitude IS NULL;
SELECT MIN(date_time), MAX(date_time) FROM accidents;
SELECT DISTINCT acc_type FROM accidents;
```

✓ **Faza 2 završena kada:** Svi xlsx fajlovi su u bazi, nema null koordinata, datumi su ispravno parsirani.

Faza 3 — EDA (Exploratory Data Analysis)

Cilj: Razumeti podatke i pronaći zanimljive obrasce.

3.1 Osnovna analiza

U `notebooks/EDA.ipynb` :

- `df.shape` , `df.dtypes` , `df.isna().sum()` — ne `.count()` !
- Distribucija po `acc_type` — koji tip je najčešći?
- Distribucija po `severity` — koliko je sa povređenima vs. materijalnom štetom?

3.2 Vremenska analiza

Parsirati `date_time` i izvući:

- `hour` — u koje doba dana se dešava najviše nezgoda?
- `day_of_week` — koji dan u nedelji?
- `month` — postoji li sezonalitet?
- Vizualizovati sa `matplotlib` bar chart-ovima

3.3 Geografska analiza

- Top 10 opština po broju nezgoda
- Top 10 opština po broju sa povređenima
- Provera koordinata — sve unutar Srbije (lat 42–46, lon 19–23)?

3.4 Korelacije

- Da li vreme dana utiče na ozbiljnost?
- Da li tip nezgode korelira sa brojem vozila?

3.5 Zaključci

Svaka sekcija u notebooku treba da ima markdown ćeliju sa nalazom. Primer: *"67% nezgoda dešava se između 7h i 19h, sa pikom u 17h što odgovara popodnevnom rush hour-u."*

✓ **Faza 3 završena kada:** Notebook ima najmanje 8 vizualizacija i svaka ima tekstualni zaključak.

Faza 4 — Backend API

Cilj: Podaci dostupni kroz REST API.

4.1 Database konekcija

U `backend/app/db/database.py` :

- `create_engine` iz `DATABASE_URL`
- `SessionLocal` i `get_db` dependency

4.2 Schemas

U `backend/app/schemas/accident.py` :

- `AccidentResponse` — šta API vraća
- `AccidentList` — lista sa `total` , `page` , `items`
- `StatsResponse` — za analitičke endpointe

4.3 Service funkcije

U `backend/app/services/accident_service.py` :

- `get_accidents(db, city, severity, page, page_size)`
- `get_accident_by_id(db, id)`
- `get_stats_by_municipality(db)`
- `get_stats_by_hour(db)`
- `get_stats_by_type(db)`

4.4 Rute

U `backend/app/api/accidents.py` :

```
GET /api/v1/accidents          - lista sa filterima
GET /api/v1/accidents/{id}     - jedan po ID
GET /api/v1/accidents/stats/municipality
GET /api/v1/accidents/stats/hour
GET /api/v1/accidents/stats/type
```

4.5 Main + health check

```
GET /health → {"status": "ok", "db": "connected"}
```

✓ **Faza 4 završena kada:** Svi endpointi rade, testirani na `http://localhost:8000/docs` .

Faza 5 — Prediktivni model

Cilj: Predviđanje ozbiljnosti nezgode (materijalna šteta vs. povređeni).

5.1 Feature engineering

Iz postojećih podataka izvuci:

- `hour` , `day_of_week` , `month` (iz `date_time`)
- `is_weekend` (bool)
- `acc_type` enkodiran kao broj
- `municipality` enkodiran kao broj

5.2 Target varijabla

`severity` → binarna klasifikacija:

- `0` = samo materijalna šteta
- `1` = povređeni ili poginuli

5.3 Treniranje

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = RandomForestClassifier(n_estimators=100)
model.fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test)))
```

5.4 Evaluacija

- Accuracy, Precision, Recall, F1
- Confusion matrix vizualizacija
- Feature importance — koji faktori najviše utiču?

5.5 Servis za predikciju

U backend/app/services/prediction_service.py :

- Učitaj model pri startu (`joblib.load`)
- Funkcija `predict_severity(hour, day, acc_type, municipality)`

5.6 Prediction endpoint

```
POST /api/v1/predict
Body: {"hour": 17, "day_of_week": "Friday", "acc_type": "...", "municipality": "..."}
Response: {"severity": "Sa povredjenim", "probability": 0.73}
```

✓ **Faza 5 završena kada:** Endpoint vraća predikciju sa verovatnoćom, F1 score > 0.70.

Faza 6 — Finalizacija & GitHub

Cilj: Projekat spreman za portfolio.

6.1 README.md

Obavezne sekcije:

- Šta projekat radi (2-3 rečenice)
- Screenshot ili GIF demonstracije
- Tech stack
- Kako pokrenuti lokalno (korak po korak)
- Gde skinuti podatke (link na data.gov.rs)

- Arhitektura sistema (dijagram)
- Ključni nalazi iz EDA

6.2 Docker za backend

Dodaj `Dockerfile` za FastAPI app i dopuni `docker-compose.yml` da pokreće i backend.

6.3 Finalna provera pre pusha

- ☐ `.env` nije na GitHubu
- ☐ `data/raw/` nije na GitHubu
- ☐ `requirements.txt` je ažuran
- ☐ Svi endpointi rade
- ☐ Notebook ima outpute (pokrenut do kraja)
- ☐ README ima instrukcije za pokretanje

Redosled prioriteta

Faza 1 → Faza 2 → Faza 3 → Faza 4 → Faza 5 → Faza 6
Setup Podaci EDA API ML GitHub